

Builder Pattern for Travel Packages

Java implementation and design report

Introduction

This project implements the Builder design pattern for a travel package. The same construction steps produce an immutable **TravelPackage** object and a readable itinerary as a **String**. A Director supplies two reusable configurations, while the client selects the representation by choosing a builder.

The example separates trip configuration from the format of the result. It demonstrates fluent method chaining, validation at the build boundary, defensive copying and dependencies expressed through an interface. Destinations and activities are demonstration data; the project does not book real travel.

Product and configurations

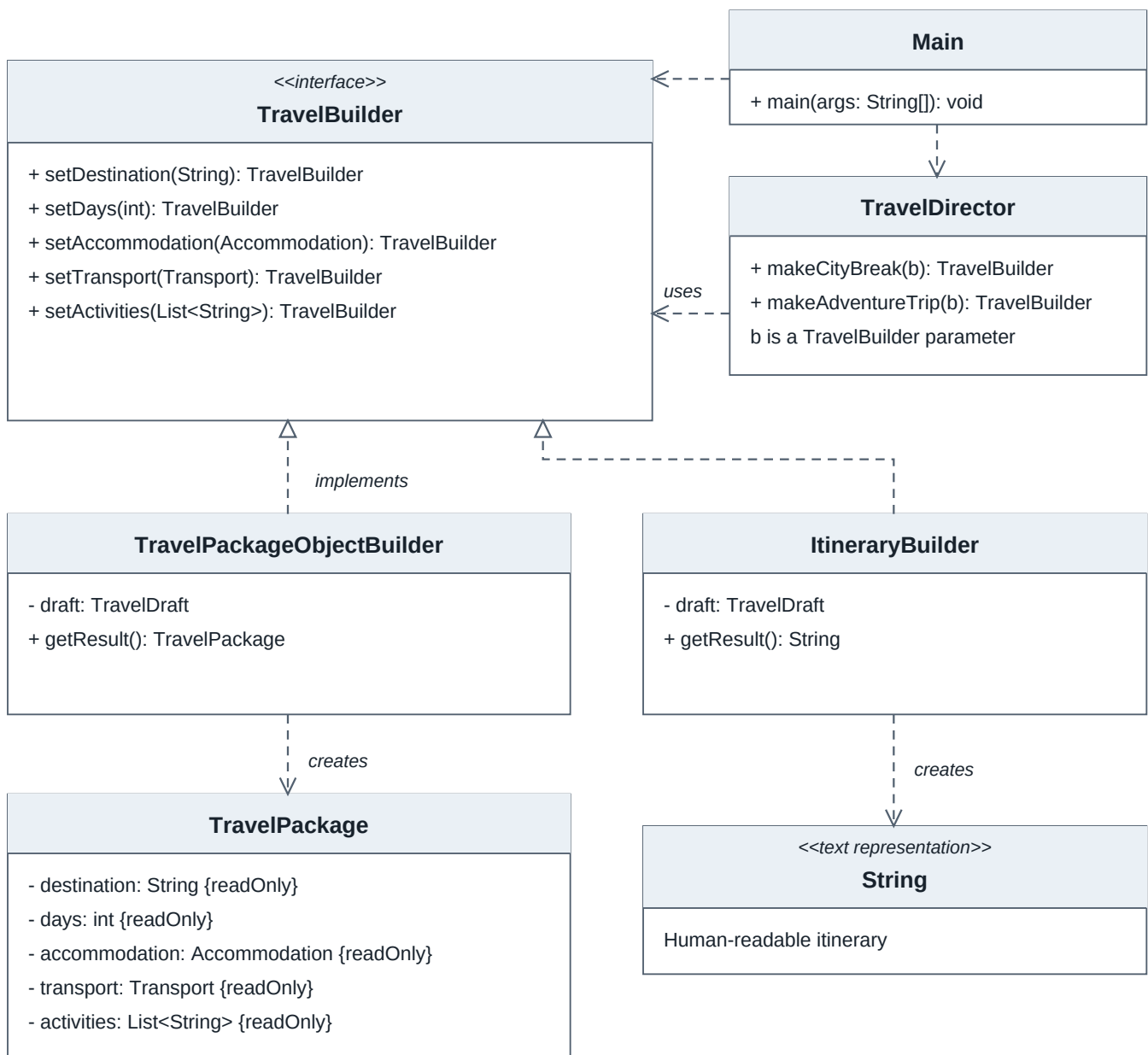
A package contains a destination, duration in days, accommodation, transport and an ordered activity list. Both presets execute the same five setters in the same order, including replacement of the complete activity list.

Attribute	City break	Adventure trip
Destination	Prague	Almaty
Duration	3 days	5 days
Accommodation	Hotel	Guesthouse
Transport	Train	Minibus
Activities	Old Town walking tour; Prague Castle visit	Mountain hike; Lake visit

Construction roles

Product: TravelPackage. **Builder:** TravelBuilder. **Concrete builders:** TravelPackageObjectBuilder and ItineraryBuilder. **Director:** TravelDirector. **Client:** Main. The following diagram shows their relationships; later sections connect Clean Code principles to actual source excerpts.

UML class diagram



Notation: + public, - private. A dashed arrow with a hollow triangle denotes interface implementation. Other dashed arrows denote dependencies or creation. {readOnly} denotes final product fields.

Each concrete builder owns a separate package-private **TravelDraft** containing its mutable state and shared validation. Accommodation and Transport are enums. These supporting types, routine getters and repeated setter signatures are omitted from the diagram. Both concrete builders implement all five steps with their own class as the return type.

getResult() belongs to each concrete builder because the return types are unrelated: TravelPackage and String. The Director uses only TravelBuilder and therefore never retrieves or casts either result.

Clean Code principles in the implementation

1 Meaningful names and named values

Names such as `makeCityBreak`, `setDestination` and `getActivities` describe their purpose. Preset data uses named constants; enum values express valid transport and accommodation choices. These names make the recipe understandable at the call site.

`TravelDirector.java`

```
private static final String CITY_DESTINATION = "Prague";
private static final int CITY_DAYS = 3;
private static final Accommodation CITY_ACCOMMODATION = Accommodation.HOTEL;
private static final Transport CITY_TRANSPORT = Transport.TRAIN;
```

2 Small methods with one clear operation

Each setter changes one aspect of the draft and returns the same builder. The concrete return type preserves fluent chaining and still satisfies the interface.

`TravelPackageObjectBuilder.java`

```
public TravelPackageObjectBuilder setDays(int days) {
    draft.setDays(days);
    return this;
}
```

3 Single responsibility

`TravelPackageObjectBuilder` creates the domain object. `ItineraryBuilder` formats text. `TravelDirector` chooses preset values. A formatting change therefore does not require changing the Director or the immutable product.

`TravelPackageObjectBuilder.java`

```
public TravelPackage getResult() {
    draft.validate();
    return new TravelPackage(draft);
}
```

Clean Code principles continued

4 Depend on an abstraction

The Director accepts TravelBuilder, so one recipe works with either concrete builder. Adding another representation requires implementing the common steps; the existing recipe remains usable without inspecting concrete classes.

TravelDirector.java

```
public TravelBuilder makeCityBreak(TravelBuilder builder) {
    Objects.requireNonNull(builder, "Builder is required.");
    return builder.setDestination(CITY_DESTINATION)
        .setDays(CITY_DAYS)
        .setAccommodation(CITY_ACCOMMODATION)
        .setTransport(CITY_TRANSPORT)
        .setActivities(CITY_ACTIVITIES);
}
```

5 Keep shared rules in one place

Both getResult methods call TravelDraft.validate(). Destination, duration, accommodation, transport and activity checks therefore use one implementation. This avoids one representation accepting a state that the other rejects.

TravelDraft.java

```
void validate() {
    if (destination == null || destination.isBlank()) {
        throw new IllegalStateException("Destination must not be blank.");
    }
    if (days < MINIMUM_DAYS) {
        throw new IllegalStateException("Duration must be at least " + MINIMUM_DAYS + " day.");
    }
    // Remaining checks omitted in this excerpt.
}
```

6 Encapsulate state and protect invariants

TravelPackage is final, its fields are private final and its constructor has package visibility. The constructor copies the activity list. Strings, enums and the day count are immutable values; the list cannot be changed through its getter.

TravelPackage.java

```
TravelPackage(TravelDraft draft) {
    draft.validate();
    destination = draft.destination();
    days = draft.days();
    accommodation = draft.accommodation();
    transport = draft.transport();
    activities = List.copyOf(draft.activities());
}
```

Validation and execution

7 Make errors explicit

getResult validates the complete draft before returning a result. Invalid or missing required values produce IllegalStateException with a field-specific message. Main catches a deliberately invalid build and prints its explanation.

TravelDraft.java

```
void validate() {
    if (destination == null || destination.isBlank()) {
        throw new IllegalStateException("Destination must not be blank.");
    }
    if (days < MINIMUM_DAYS) {
        throw new IllegalStateException("Duration must be at least " + MINIMUM_DAYS + " day.");
    }
    if (accommodation == null) {
        throw new IllegalStateException("Accommodation is required.");
    }
    if (transport == null) {
        throw new IllegalStateException("Transport is required.");
    }
    if (activities == null || activities.isEmpty()) {
        throw new IllegalStateException("At least one activity is required.");
    }
    for (String activity : activities) {
        if (activity == null || activity.isBlank()) {
            throw new IllegalStateException("Activities must not contain null or blank entries.");
        }
    }
}
```

Running the project

Install JDK 17 or newer. No Maven, Gradle or third-party Java libraries are required. From the project directory, run:

```
./run.sh
./test.sh
```

The scripts compile the code with --release 17. Main demonstrates both presets in both representations, a custom fluent configuration and a validation error.

Example itinerary

```
TRAVEL ITINERARY
Destination: Prague
Duration: 3 days
Accommodation: Hotel
Transport: Train
Activities:
- Old Town walking tour
- Prague Castle visit
```

Verification and conclusion

Verification results

The executable test runner was compiled and run with the supplied test.sh script. The following is the recorded result:

```
PASS: All fluent steps return the same builder
PASS: Both builders reject missing and invalid fields
PASS: City break has matching object and text values
PASS: Adventure trip has matching object and text values
PASS: Director uses the same five interface steps
PASS: Mutable input lists are copied by both builders
PASS: Product activities cannot be modified
PASS: Builder reuse preserves old products and replaces activities
PASS: Destination and activity whitespace is normalized
PASS: Every getResult validates current state
PASS: Product exposes no public construction or mutable fields
PASS: Concrete fluent chains expose the right result types
All 12 tests passed.
```

Property	What the executable checks establish
Two representations	Both Director presets produce matching destination, duration, accommodation, transport and activities.
Fluent interface	Every construction step returns the same concrete builder instance.
Validation	Both builders reject missing or invalid required state with clear exceptions.
Immutability	Changing the input list or reusing the builder does not change an already built product; its activity list rejects mutation.
Reuse	Applying another preset replaces the old activity list and the remaining fields.

Conclusion

The implementation meets the Builder requirements by sharing construction steps across two result types. The Director stores reusable recipes, each builder produces its own representation, and the immutable product is isolated from subsequent draft changes. Shared validation keeps the accepted input rules consistent. The additional classes and setter delegation add some code, but make the two representations and their responsibilities explicit.

Repository and submission

GitHub repository: https://github.com/Gakonn/Builder_Pattern

The source package includes a short README, the Java implementation, a test runner, this report, editable UML source and Russian notes for the in-class defense.